Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sift-Up y Sift-Down: las dos operaciones que reparan la propiedad de heap
- Insertar y Extraer-Extremo, completos
- Construir-Heap en $O(n)$ — la demostración prometida la clase pasada

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La clase pasada

Vimos la propiedad de heap, su representación con arreglo (fórmulas *padre*, *izq*, *der*, válidas tanto 0-indexado como 1-indexado), y prometimos que Insertar y Extraer-Extremo son $O(\lg n)$, y que Construir-Heap es $O(n)$ — sorprendentemente, no $O(n \lg n)$. Hoy implementamos todo con la convención **1-indexado** que elegimos la clase pasada ($izq(i) = 2i$, $der(i) = 2i + 1$, $padre(i) = \lfloor i/2 \rfloor$).

Insertar al final del arreglo rompe la propiedad de heap — ¿cómo la reparas, sin recorrer todo el arreglo?

Insertar un elemento al final del arreglo es $O(1)$, pero probablemente **rompe** la propiedad de heap.

Sift-Up: reparando hacia la raíz

La idea

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Solo un camino puede estar roto

Si insertamos x al final del arreglo (siguiente posición libre del árbol completo), la **única** violación posible de la propiedad de heap es entre x y su padre — todo el resto del árbol seguía siendo válido antes de insertar.

Si x es menor que su padre, **intercambiarlos** arregla esa violación — pero puede crear una nueva, un nivel más arriba. Repetir mientras haga falta: eso es SIFT-UP.

Sift-Up, en código

Algoritmo 3: SIFT-UP(H, i)

mientras $i > 1$ ***y*** $H[padre(i)] > H[i]$ **hacer**
    intercambiar $H[padre(i)]$ con $H[i]$
    $i \leftarrow padre(i)$

¿Cuántas iteraciones puede hacer este `while`, como máximo?

Respuesta

A lo más la **altura** del árbol: $O(\lg n)$. Cada iteración sube un nivel, y el árbol tiene $O(\lg n)$ niveles por ser completo.

Insertar

Insertar, completo

Algoritmo 5: INSERTAR(H, x)

$n \leftarrow tamao(H)$
$H[n+1] \leftarrow x$
$tamao(H) \leftarrow n+1$
SIFT-UP$(H, n+1)$

Colocar al final es $O(1)$ (siempre es la siguiente posición libre del árbol completo, por construcción); SIFT-UP es $O(\lg n)$. **Total:** $O(\lg n)$, tal como prometimos la clase pasada.

Sift-Down: reparando hacia las hojas

La idea

EXTRAER-EXTREMO deja un "hueco" en la raíz — ¿con qué lo llenamos, para no romper la estructura de árbol completo?

El mínimo está en la raíz — fácil de quitar — pero eso deja un hueco.

Con el último elemento

Mover el **último** elemento del arreglo a la raíz preserva la forma de árbol completo (quitamos exactamente la última posición) — pero casi seguro rompe la propiedad de heap **hacia abajo**.

Reparando hacia abajo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La idea de Sift-Down

Mientras el nodo actual sea **mayor** que alguno de sus hijos, intercambiarlo con el **menor** de los dos (¡importante: el menor, no cualquiera!) y continuar desde ahí.

¿Por qué hay que intercambiar con el **menor** de los dos hijos, y no simplemente con cualquiera de ellos que sea menor que el nodo actual?

Respuesta

Si intercambiáramos con el hijo **mayor** (siendo aún menor que el nodo actual), el hijo **menor** podría quedar como nuevo padre de un valor más grande que él mismo — violando la propiedad de heap otra vez, sin haberla realmente reparado.

Sift-Down, en código

Algoritmo 6: SIFT-DOWN(H, i, n)

mientras *verdadero* **hacer**
- $m \leftarrow i$
- **si** $izq(i) \leq n$ **y** $H[izq(i)] < H[m]$ **entonces**
  - $m \leftarrow izq(i)$
- **si** $der(i) \leq n$ **y** $H[der(i)] < H[m]$ **entonces**
  - $m \leftarrow der(i)$
- **si** $m = i$ **entonces**
  - **romper**
- intercambiar $H[i]$ con $H[m]$
- $i \leftarrow m$

Extraer-Extremo

Extraer-Extremo, completo

Algoritmo 8: EXTRAER-EXTREMO(H)

$n \leftarrow tamao(H)$
$min \leftarrow H[1]$
$H[1] \leftarrow H[n]$
$tamao(H) \leftarrow n - 1$
SIFT-DOWN($H, 1, n-1$)
devolver min

Guardamos el mínimo antes de sobrescribirlo, movemos el último al frente ($O(1)$), y reparamos con SIFT-DOWN ($O(\lg n)$). **Total:** $O(\lg n)$.

Construir-Heap en $O(n)$

El enfoque ingenuo

¿Qué complejidad da empezar con un heap vacío e INSERTAR los n elementos uno por uno?

$O(n \lg n)$

n inserciones, cada una $O(\lg n)$ en el peor caso. Funciona, pero la clase pasada prometimos que se puede hacer mejor: $O(n)$.

Un enfoque distinto: de abajo hacia arriba

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La idea

En vez de insertar uno por uno, coloca los n elementos **directamente** en el arreglo (en cualquier orden), y luego aplica SIFT-DOWN a cada nodo **interno** — empezando por el **último** y terminando en la raíz.

¿Por qué hay que ir de **último a primero**, y no al revés?

Respuesta

SIFT-DOWN(H, i, n) asume que los **subárboles** de i ya son heaps válidos. Procesando de abajo hacia arriba, cuando le toca a i, sus hijos — procesados antes — ya cumplen la propiedad.

Construir-Heap, en código

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 10: CONSTRUIR-HEAP(H, n)

para $i \leftarrow \lfloor n/2 \rfloor$ **a** 1 ***(decreciente)*** **hacer**
  SIFT-DOWN(H, i, n)

Empezamos en $\lfloor n/2 \rfloor$: es el **último** nodo con al menos un hijo (los nodos $\lfloor n/2 \rfloor + 1, \ldots, n$ son hojas — ya son heaps triviales, no hace falta procesarlos).

¿Por qué esto es $O(n)$, y no $O(n \lg n)$?

Eso ya suena a $O(n \lg n)$ — ¿dónde está la ganancia?

Llamamos a SIFT-DOWN unas $n/2$ veces, cada una cuesta $O(\lg n)$ en el peor caso.

La clave: no todos los nodos están igual de "altos"

SIFT-DOWN(H, i, n) no cuesta $O(\lg n)$ siempre — cuesta $O(h_i)$, donde h_i es la **altura** del nodo i dentro del árbol. Y la **mayoría** de los nodos están cerca de las hojas, con altura pequeña.

La cuenta completa

En un árbol completo con n nodos, hay a lo más $\lceil n/2^{h+1} \rceil$ nodos de altura h. El costo total es entonces:

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right)$$

Y $\sum_{h=0}^{\infty} h/2^h = 2$ (serie geométrica derivada, convergente) — así que la suma completa es $O(n \cdot 2) = O(n)$.

La intuición, sin la suma

La mitad de los nodos son hojas ($h = 0$, costo 0); un cuarto tiene altura 1 (costo $O(1)$ cada uno); un octavo tiene altura 2; y así sucesivamente. **Muy pocos** nodos (cerca de la raíz) pagan el costo alto $O(\lg n)$ — y son exponencialmente pocos. Esa es la razón real por la que Construir-Heap es $O(n)$: **no** es que cada Sift-Down sea barato, es que los caros son rarísimos.

Heapsort: un adelanto

La idea, en una diapositiva

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Heapsort

Construir-Heap(H, n) en $O(n)$; luego, n veces: intercambiar $H[1]$ con el último elemento activo, reducir el tamaño del heap en 1, y Sift-Down desde la raíz. Cada extracción coloca el máximo restante en su posición final — ordenamiento *in-place*, $O(n \lg n)$ total.

No lo implementamos hoy a fondo — solo como evidencia de que todo lo construido esta sesión (Sift-Down, Construir-Heap) ya alcanza para un algoritmo de ordenamiento completo.

Cierre de la sesión

- SIFT-UP repara **un solo camino** hacia la raíz tras insertar — $O(\lg n)$, usado por INSERTAR.
- SIFT-DOWN repara hacia las hojas, siempre intercambiando con el **menor** de los hijos — $O(\lg n)$, usado por EXTRAER-EXTREMO.
- CONSTRUIR-HEAP aplica SIFT-DOWN de abajo hacia arriba, solo en nodos internos — $O(n)$, no $O(n \lg n)$, porque casi todos los nodos tienen altura pequeña.
- Con estas piezas, HEAPSORT — un ordenamiento $O(n \lg n)$ *in-place* — sale casi gratis.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos implementar Sift-Up y Sift-Down, y explicar por qué cada uno es $O(\lg n)$.
- Sabemos implementar Insertar y Extraer-Extremo completos.
- Sabemos implementar Construir-Heap y demostrar por qué es $O(n)$, no $O(n \lg n)$.
- Entendemos cómo estas piezas se combinan para dar Heapsort.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias